SmartPOS

Project Guide

Point-of-Sale and Inventory Management System

Directory structure, every module and file, how and why it was built, what is done and what is left.

Stack: Angular 22 · ASP.NET Core 10 · Entity Framework Core 10 · PostgreSQL 16 · JWT
Status: working foundation, verified end to end (updated 28 October 2026)
Revision note: this edition replaces the first one. Since then the database moved from MySQL to PostgreSQL, the app became multi-shop with one kind of account, and the dashboard, loyalty points and email verification were added.

1. What SmartPOS is

SmartPOS is a retail point-of-sale (POS) and inventory system for small shops. A person registers, which creates their account and their shop. They add products with stock, register customers, and sell: the cashier screen searches products, builds a cart, applies a discount, takes cash, card or UPI payment (with change for cash), optionally attaches a customer who earns loyalty points, and prints a bill. The system prices every sale itself, deducts stock, and records every stock change. A dashboard turns the sales history into KPIs and charts.

This repository is the **foundation** the team will extend. Everything described here is real, connected code against a real database. Offline billing and AI features were explicitly out of scope and are not built or claimed.

Technology choices and why

Technology	Role and reason
ASP.NET Core 10 Web API	The only .NET SDK on the machine is 10, so every package had to support it. Controllers stay thin; logic lives in services that can be unit-tested without HTTP.
EF Core 10 + Npgsql (PostgreSQL provider 10.0.3)	EF Core gives versioned migrations and LINQ. The project started on MySQL and was moved to PostgreSQL on request; Npgsql is the mature provider and maps dates, timestamps with time zone, decimals and identity columns natively.
PostgreSQL 14+ (verified on 16)	Strict, standards-following SQL with CHECK constraints (used as a last defence against negative stock) and row-level locking for the stock update.
JWT bearer tokens	Stateless login for a single-page app. The token carries the user id and the shop id; every request also re-checks the user in the database.
ASP.NET Identity PasswordHasher	A maintained PBKDF2 implementation with a per-password salt and automatic re-hash. Only the hasher is used, not the whole Identity user store.
Angular 22 (standalone, signals, zoneless)	Current conventions: no NgModules, signals for state, OnPush everywhere, lazy-loaded routes, native <dialog> modal.
Hand-written SVG charts	The dashboard charts (area, bars, donut, heatmap, sparkline, count-up numbers) are small Angular components with no chart library, so the bundle stays small and the look matches the app.
Plain CSS design system	One stylesheet of design tokens and shared classes: charcoal sidebar, green accent, a validated blue-led chart palette.
Google Public DNS (over HTTPS)	A free, key-less way to ask whether an email domain can receive mail. Used at sign-up; only the domain is sent.
xUnit, SQLite, Vitest	xUnit for backend tests; in-memory SQLite lets API integration tests run without a database server; Vitest ships with Angular 22.

How to read this guide

- Section 2 is the story of what was built, in order. Section 3 is the directory tree.
- Sections 4-8 explain the backend layer by layer; Section 9 the database; Section 10 the frontend.
- Sections 11-14 explain four ideas in depth: multi-shop isolation, loyalty points, email verification, the sale flow.
- Section 15 lists problems found along the way, 16 what was verified, 17 what is left, 18 how to run it.

2. What we did, in order

Phase	What happened
1. Foundation	Inspected the machine, then built the layered solution (Core, Application, Infrastructure, API plus two test projects), the Angular app, and the first schema: users, categories, products, sales, sale items, stock movements. JWT login with Admin and Cashier roles, POS checkout, inventory, reports, user management. Verified against a throwaway MySQL 9.6.
2. Schema redesign (multi-shop)	Replaced the schema with the requested design: Sales and SaleItems columns exactly as specified (BillNumber, UserId, TotalAmount, AmountPaid, ChangeAmount, Status, per-line discount and tax) plus the Shop, Customer, Product and Inventory models. Added a ShopId to every shop-owned table and a global query filter so shops cannot see each other's data. Renames were applied through the API and the Angular app.
3. One account type, PostgreSQL	Removed the Admin/Cashier roles: each user owns exactly one shop and registers themselves (self sign-up creates the user and the shop). Removed the user-management screen and the bootstrap admin. Switched the database from MySQL to PostgreSQL (provider, migration, CHECK constraints now quoted, case-insensitive search).
4. Analytics dashboard + demo history	Added an insights endpoint and a dashboard with KPIs compared with the previous period, a revenue trend, payment mix, category sales, busiest hours, a weekday-by-hour heatmap, top products and customers, with animations. Added a development-only generator of fabricated sales history, and a command to backfill it into an existing shop without touching real records.
5. Loyalty points	Bills with a customer earn points by a configurable rule; points are stored on the bill and added to the customer's balance atomically. The POS previews and confirms them. Existing and demo data were backfilled with the same rule.
6. Email validation and verification	Sign-up refuses throwaway domains and domains with no mail server (free DNS lookup), then emails a single-use confirmation link. Pluggable sender (SMTP or, in development, the log). Banner and verification page in the app.
7. Operations	A development-only command to reset a forgotten password; created the real local PostgreSQL role and database, applied the migrations, and kept the API running.

Each phase was verified before moving on, and several problems were found by verifying against the *real* database or browser rather than the in-memory test setup (Section 15).

3. Directory structure

```
smartpos/
|-- README.md                 quick start, config, features, limitations
|-- .env.example              names of environment variables (no real values)
|-- database/schema.sql       SQL generated from the EF Core migrations (reference only)
|-- docs/                     ARCHITECTURE, DATABASE, API, TESTING, this guide
|-- backend/
|   |-- SmartPOS.slnx  Directory.Build.props  Directory.Packages.props  .config/dotnet-tools.json
|   |-- src/
|   |   |-- SmartPOS.Core/            entities + enums (no dependencies)
|   |   |   |-- Entities/  Shop User Customer Product Inventory Sale SaleItem StockMovement
|   |   |   `-- Enums/     PaymentMethod SaleStatus StockMovementType
|   |   |-- SmartPOS.Application/     services, DTOs, business rules, interfaces
|   |   |   |-- Auth/       AuthService, AuthDtos, EmailAbstractions, security interfaces
|   |   |   |-- Sales/      SaleService, SaleCalculator, DiscountPolicy, SaleDtos
|   |   |   |-- Products/   Inventory/   Customers/ (+ Loyalty.cs)   Categories/
|   |   |   |-- Dashboard/  Dashboard.cs (today), Insights.cs (analytics)
|   |   |   |-- Reports/    Common/ (options, paging, errors, calendar, current user/shop)
|   |   |-- SmartPOS.Infrastructure/  database, email, security
|   |   |   |-- Persistence/  SmartPosDbContext, Configurations/, Migrations/, SampleData,
|   |   |   |                 SampleDataSeeder, DemoHistory, DemoHistoryCommand, ResetPasswordCommand
|   |   |   |-- Email/        SMTP / logging / null senders, DNS domain checker
|   |   |   `-- Security/     IdentityPasswordHasher
|   |   `-- SmartPOS.API/             Program.cs, Controllers/, Auth/, Infrastructure/
|   `-- tests/  SmartPOS.UnitTests (63)   SmartPOS.IntegrationTests (51, SQLite, two shops)
`-- frontend/src/
    |-- styles.css                    design tokens + shared classes + chart motion
    `-- app/
        |-- core/      auth, api/*.service.ts, models.ts, http helpers, toast/confirm
        |-- shared/    components/ (icon, modal, states...), pipes/, charts/ (SVG toolkit)
        |-- layout/    shell (with verify banner), sidebar, header
        `-- features/  auth (login, register, verify-email), dashboard, products, customers,
                       inventory, pos, sales, reports
```

Dependency direction (the most important rule)

```
SmartPOS.API  ->  SmartPOS.Infrastructure  ->  SmartPOS.Application  ->  SmartPOS.Core
```

Arrows mean "may reference". Core knows nothing else. Application defines *what* the system does and the interfaces it needs (for example `IEmailSender`, `IPasswordHasher`); Infrastructure supplies the real database, mail and hashing; API is the outermost shell. Because dependencies only point inwards, business rules are tested without a web server or a database, and the email sender can be swapped for a fake in tests.

4. SmartPOS.Core: the domain

Plain C# classes and enums with no framework dependencies. They describe *what data exists*.

Entity	Fields and why
Shop	ShopName, Address, CreatedAt. The tenant: every other table (except Shops) carries a ShopId. A shop has exactly one user, its owner.
User	The login. ShopId, FullName, Email (stored lower-case, unique across all shops because it is the login name), PasswordHash (PBKDF2), EmailVerifiedAt, plus the SHA-256 hash and expiry of a pending verification link. **No role column:** there is one kind of account.
Customer	ShopId, Name, Phone (unique within the shop), LoyaltyPoints (the balance), CreatedAt.
Product	ShopId, Sku (unique per shop, stored upper-case), Barcode (unique per shop when present), Name, Description, Category (plain text), Price, CostPrice, ExpiryDate (date), IsActive. Stock is deliberately *not* here.
Inventory	One row per product: StockQuantity, LowStockThreshold (default 20), LastUpdated. Changed only by sales and adjustments, never by editing a product, so every change has a ledger entry.
Sale	BillNumber (unique), ShopId, UserId, optional CustomerId, Subtotal, DiscountPercent, DiscountAmount, TaxRatePercent (the rate used, so old bills reproduce), TaxAmount, TotalAmount, PaymentMethod, AmountPaid, ChangeAmount, LoyaltyPointsEarned, Status, CreatedAt.
SaleItem	A snapshot of the line at sale time: ProductName, Sku, Quantity, UnitPrice, UnitCost, plus this line's DiscountAmount, TaxAmount and LineTotal (final amount). Later product edits never change old bills or reports.
StockMovement	Append-only ledger: QuantityChange, QuantityAfter, Type (InitialStock, Sale, Adjustment), Reason, optional SaleId and UserId.
Enums	PaymentMethod (Cash, Card, UPI, Other), SaleStatus (Pending, Completed, Cancelled), StockMovementType. Stored as readable text in the database.

Why: Prices and costs are copied onto each line, stock lives in its own table, and the owner is just the shop's one user. Each choice keeps history stable and stops a single mistake (a price edit, a stock typo) from rewriting the past.

5. SmartPOS.Application: services and rules

Everything the system *does*. Services talk to the database through `IApplicationDbContext`, a small interface exposing EF Core DbSets plus one carefully written operation, `TryChangeStockAsync`. A generic repository per entity was rejected: it would wrap LINQ in a weaker copy of itself.

Sales

File	What it does / why
SaleCalculator.cs	The single source of truth for sale arithmetic. Line subtotal = price x quantity; discount comes off the subtotal first; tax is charged on the discounted amount; everything rounds half away from zero to 2 decimals. The discount and tax are then **split across the lines to the cent** (largest-remainder method) so the lines always add up exactly to the bill. A naive rounding of each share could leave a negative cent; this cannot. `Settle()` works out amount paid and change: cash may be over-tendered, other methods must be exact.
SaleService.cs	Creates a sale. Merges duplicate lines, sorts by product id (a consistent lock order prevents deadlocks), loads prices **from the database, never the client**, checks stock and the discount limit, calculates totals, then in one transaction: conditionally decrements stock per line, inserts the sale, items and stock movements, and adds loyalty points. Any failure rolls everything back.
DiscountPolicy.cs	One configurable limit (`Pos:MaxDiscountPercent`); a bigger discount returns 403.
SaleDtos.cs	What the client sends (items, discount %, payment method, optional amount paid, optional customer) and gets back. Notice what is *not* sent: prices and totals.

Stock, products, customers

File	What it does / why it exists
StockRules.cs	Friendly pre-checks (enough stock, valid adjustment, status). The database update remains the authoritative guard.
InventoryService.cs	Lists stock sorted by urgency, applies manual adjustments (always with a reason) and lists the movement ledger.
ProductService.cs	Search (name, SKU, barcode, case-insensitive), create (also creates the inventory row and opening movement in one save), update, delete only if never sold. SKU and barcode uniqueness are per shop.
CustomerService.cs	Create, update, search by name or phone. Phones are normalised (spaces and dashes stripped) so the same number is never stored twice.
Categories.cs	A category is just the text on a product, so this lists the distinct names in use with counts.
Loyalty.cs	`LoyaltyRules.PointsEarned`: the earning rule, explained in Section 12.

Auth, dashboard, reports

File	What it does / why it exists
AuthService.cs	Login (generic error, dummy hash check for unknown emails so timing does not reveal who has an account), registration (email checks, creates shop and user in one transaction, sample data, verification email), email verification and resend. See Sections 11 and 13.
EmailAbstractions.cs	`IEmailSender`, `IEmailAddressChecker` and the list of throwaway email domains.
Dashboard.cs	Today's totals, counts, recent bills and low-stock products.
Insights.cs	The analytics behind the dashboard charts: one narrow query for two periods, bucketed in memory by store-local day, weekday and hour. Returns KPIs with previous-period figures, a daily series, a 7 x 24 heatmap, payment mix, categories, top products and top customers. Completed bills only.

File	What it does / why it exists
Reports.cs	Daily sales and top products over a date range (max 366 days), net of tax.
Common/	AppExceptions (mapped to HTTP codes), paging, the store calendar (UTC in the database, store time zone for days and hours), options classes (Store, Pos, Loyalty, App), SearchPattern (lower-cases and escapes LIKE input), current user and shop interfaces.

Key code: the sale arithmetic

```
var subtotal = lineSubtotals.Sum();
var discount = Round(subtotal * discountPercent / 100m);
var tax      = Round((subtotal - discount) * taxRatePercent / 100m);   // tax AFTER discount
var lineDiscounts = Allocate(discount, lineSubtotals);                  // to the cent, sums exactly
var lineTaxes     = Allocate(tax, lineSubtotals - lineDiscounts);
// line total = subtotal - discount + tax;  the lines add up to subtotal - discount + tax
```

6. SmartPOS.Infrastructure: database, email, seeding

Persistence

File	What it does / why it exists
SmartPosDbContext.cs	The EF Core context. Adds the **tenant query filter** (Section 11) to every shop-owned entity, keeps all timestamps UTC (Npgsql requires it), and implements `TryChangeStockAsync`, the atomic conditional stock update.
Configurations/*.cs	One class per entity: lengths, money precision, enum-as-text, indexes (unique per shop for SKU, barcode, phone), foreign keys with RESTRICT or CASCADE as appropriate, and CHECK constraints. In PostgreSQL the column names inside a CHECK must be **quoted** or they are folded to lower case and fail.
Migrations/	Two migrations: `InitialCreate` (the whole schema) and `AddLoyaltyAndEmailVerification` (new columns plus a backfill of points for existing bills). Applied explicitly with `dotnet ef database update`, never silently at startup.
SampleData.cs, SampleDataSeeder.cs	A fictional 16-product catalogue given to each newly registered shop in Development, with stock and opening movements.
DemoHistory.cs	Fabricated sales history for demos (Section 9.3).
DemoHistoryCommand.cs	Backfills that history into an existing shop. **ResetPasswordCommand.cs** sets a new password. Both are development-only command-line tools.

Key code: the atomic stock update

```
// UPDATE "Inventories" SET "StockQuantity" = "StockQuantity" + @delta ...
// WHERE "ProductId" = @id AND "StockQuantity" >= -@delta        (and the shop filter)
var affected = await Inventories
    .Where(i => i.ProductId == productId && i.StockQuantity >= -delta)
    .ExecuteUpdateAsync(s => s.SetProperty(i => i.StockQuantity, i => i.StockQuantity + delta)
                              .SetProperty(i => i.LastUpdated, now));
return affected == 0 ? null : newQuantity;   // null = would go negative -> the sale rolls back (409)
```

The check and the write are one statement, and the row lock is held until the transaction ends, so two cashiers can never both take the last unit. A CHECK constraint (stock >= 0) is the final safety net.

Email

Class	What it does
SmtpEmailSender	Sends through any SMTP server (settings under Email:Smtp; the password lives in user-secrets). Free choices: Gmail with an app password, Brevo or Mailjet free tiers, a local Mailpit.
LoggingEmailSender	Development stand-in when no SMTP host is set: prints the whole email, including the verification link, to the API log. The link is a credential, so this is never on by default.
NullEmailSender	Nothing configured: says so in the log without printing the message.
DnsOverHttpsEmailAddressChecker	Asks Google Public DNS for the domain's mail servers (MX, falling back to an address record). Definitive answers are cached for an hour. A failed lookup returns Unknown and never blocks sign-up.

Security/IdentityPasswordHasher.cs wraps the Identity hasher behind the application's own interface, including a dummy verification that keeps login timing similar for unknown emails.

7. SmartPOS.API: the HTTP layer

File	What it does / why it exists
Program.cs	Wires the layers, exception handler, Swagger (Development only), CORS, authentication, rate limiting, health endpoint, and the two command-line modes (`seed-demo-history`, `reset-password`) which refuse to run outside Development.
Auth/	JwtTokenService issues tokens with sub, name, email, shop and jti claims; CurrentUser and CurrentShop read them for the services and the database filter.
Infrastructure/ApiServiceCollectionExtensions.cs	JWT validation plus a per-request check that the user still exists and still belongs to the token's shop; the fallback policy that requires sign-in everywhere unless an endpoint says otherwise; the rate limiter (10 attempts per minute per IP on login, registration, verification and resend).
GlobalExceptionHandler.cs	Turns exceptions into RFC 7807 problem responses: business-rule 400, forbidden 403, not found 404, conflict 409. Unexpected errors return a generic message and a trace id; details stay in the server log.
DatabaseHealthCheck.cs	GET /health, without exception text (which could leak host names).

Endpoints (all need a token unless marked public)

Endpoint	Notes
POST /api/auth/login, /register, /verify-email	Public, rate limited. Register creates the shop and user and emails a link.
POST /api/auth/resend-verification, GET /api/auth/me	Signed-in user.
GET /api/dashboard/summary, /insights?days=	Today's figures; analytics for 1-180 days.
GET /api/settings/pos	Shop name and address, currency, tax rate, discount limit, loyalty rate.
/api/products (GET, POST, PUT, DELETE), /api/categories	Catalogue. PUT replaces every field.
/api/customers (GET, POST, PUT)	Customers and their points.
GET /api/inventory, POST /api/inventory/adjustments, GET /api/inventory/movements	Stock and the ledger.
POST /api/sales, GET /api/sales, GET /api/sales/{id}	Checkout, history, a bill.
GET /api/reports/daily-sales, /top-products	Range reports.
GET /health	Public.

There are **no roles**: every signed-in user has the same access, to their own shop only. Anything belonging to another shop returns 404 exactly as if it did not exist.

8. Backend tests

Suite	Covers
Unit tests (63)	SaleCalculator (totals, rounding, discount-then-tax, line allocation always sums exactly, settlement and change), StockRules, DiscountPolicy, StoreCalendar (time-zone day boundaries), SearchPattern, LoyaltyRules, and DemoHistory (stock ledger ends exactly on target and never goes negative, bills consistent, deterministic, two shops never share a bill number, loyalty balances add up).
Integration tests (51)	The real HTTP pipeline on in-memory SQLite with **two shops**: registration and login, email verification (single-use, replaced and forged links, resend, throwaway and undeliverable domains, lookup failure allowed), product validation, duplicate SKU and barcode, sales pricing and stock deduction, oversell rollback, cash change, loyalty points, customers, dashboard insights, and one shop being unable to see, change or sell another's data. A fake mail sender and domain checker keep tests offline.

What SQLite cannot show: PostgreSQL's SQL dialect, case-sensitive LIKE, real locking, and CHECK-constraint quoting. Those were verified manually against real PostgreSQL (Section 16). SQLite also compares decimals as text, so a CHECK like discount <= 100 rejects 12.5 there; integration tests therefore use round percentages and the unit tests cover awkward ones.

9. The database

9.1 Tables

Table	Key rules
Shops	ShopName, Address.
Users	ShopId (unique: one user per shop), Email (unique), PasswordHash, EmailVerifiedAt, pending-link hash and expiry.
Customers	Unique (ShopId, Phone); LoyaltyPoints with CHECK >= 0.
Products	Unique (ShopId, Sku) and (ShopId, Barcode); Price and CostPrice numeric(18,2) CHECK >= 0; ExpiryDate (date).
Inventories	Unique ProductId (cascade delete); StockQuantity CHECK >= 0; LowStockThreshold.
Sales	Unique BillNumber; money columns numeric(12,2); LoyaltyPointsEarned CHECK >= 0; index (ShopId, CreatedAt).
SaleItems	FK to Sales (cascade) and Products (restrict: a sold product can never be deleted, only deactivated); Quantity CHECK > 0.
StockMovements	The ledger; QuantityAfter CHECK >= 0.

All timestamps are `timestamp with time zone`, always UTC. "Today" and date filters use the store's time zone (configuration), applied by the API, not the database.

9.2 Rules the database enforces itself

- Unique emails, SKUs per shop, barcodes per shop, bill numbers, customer phones per shop.
- No negative stock, prices, totals or loyalty points (CHECK constraints), even if application code has a bug.
- Foreign keys: you cannot delete a product that was sold, a customer that has bills, or a user or shop that owns records.

9.3 Fabricated demo history

So that dashboards have something to show, a **development-only** generator invents about 90 days of history. It is clearly fictional (16 customers with fake phone numbers like 9000000101) and is switched on only by `appsettings.Development.json`.

- It is seeded *per shop*, so a shop always gets the same history but two shops never share a bill number.
- It shapes the data realistically: weekends busier, lunch and evening peaks, slow growth, one festival-style rush, a few popular products, mostly cash and UPI, about 38% of bills with a customer, a few discounts.
- Bills go through the real SaleCalculator and the real loyalty rule.
- The stock ledger stays consistent: each product's opening stock is its sample stock plus everything that will be sold, so the sales run it down to exactly the sample level.
- `seed-demo-history --email ...` adds it to an existing shop: the history is placed before the shop's first real activity, opening-stock entries are moved back to the start, it is all-or-nothing, and it refuses to run twice. It was rehearsed on a copy of the real database first.

10. The Angular frontend

Core and layout

File	What it does / why it exists
core/auth/	AuthService (login, register, verify, resend, refresh; session in localStorage), an interceptor that attaches the token and signs out on 401, guards (signed in / guests only).
core/api/	One typed service per API area: products, customers, inventory, sales (also reports and dashboard), settings.
core/models.ts	TypeScript mirrors of the API's DTOs.
core/http, core/ui	Query-string builder, error-message mapper, toast and confirm services.
layout/	Shell (sidebar, header, routed page, and the amber banner that asks the user to confirm their email with a Resend button), sidebar navigation, header showing the shop name.

Features (lazy-loaded routes)

File	What it does / why it exists
auth/	Login; Register (shop name and address, name, email, password); the verify-email landing page for the emailed link.
dashboard/	Today's cards, then the sales-performance section (below), recent bills and low stock.
products/	List with search (name, SKU, barcode) and category filter; form with barcode, free-text category with suggestions, expiry date, low-stock level.
customers/	List, search and a modal to add or edit; shows each customer's loyalty points and the earning rate.
inventory/	Stock by urgency with filters, an adjustment dialog, and the movement history.
pos/	Product search or barcode scan with Enter, cart with quantity limits, discount, payment method (Cash, Card, UPI, Other), cash received with change due, optional customer with a live 'earns N points' preview, and a completion panel. The cart maths mirrors the server in integer cents.
sales/	History with date filters and bill-number search; the itemised bill showing each line's discount and tax, amount paid, change and points; printable.
reports/	Daily sales and top products for any range.

The dashboard and its chart toolkit

The dashboard calls `/api/dashboard/insights` for 7, 30 or 90 days and shows:

- **Six KPI cards** that count up: revenue, bills, average bill, items sold, gross profit, customers served; each with the change against the previous period (green up, red down, grey no change) and a sparkline.
- **Highlights** written from the data: best day, busiest weekday and hour, leading category, share of bills with a customer.
- **Revenue over time**: an area chart that draws itself in, with the previous period as a dashed line, a crosshair tooltip and keyboard control.
- **Payment mix** donut, **sales by category** and **top products and customers** as ranked bars, **busiest hours** as columns with the peak labelled, and a **weekday x hour heatmap** in one blue hue from quiet to busy.

All of it is plain SVG in `shared/charts/`: KpiCard, CountUp, Sparkline, AreaChart, BarChart, HBarChart, DonutChart, Heatmap and shared helpers (nice axis scales, number formatting, size observer). Colours come from a validated palette (blue first, fixed colours per payment method, one hue ramp for magnitude). Charts are accessible (summary labels, a hidden data table behind the area chart, keyboard control), and every animation switches off for users who ask for reduced motion.

Why: No chart library keeps the bundle small (the dashboard chunk is about 51 kB) and avoids a dependency to maintain. One thing found in testing: a transient zero width during layout used to tear charts down and replay their animations, so zero widths are now ignored.

11. Idea 1: multi-shop isolation

Many shops can use one database without seeing each other. Three things make that work:

- **Every shop-owned row carries a ShopId.**
- **The token carries the shop.** At login the JWT gets a `shop` claim; every request also re-checks that the user still belongs to that shop.
- **A global query filter** on the database context adds `ShopId = current shop` to every query on those tables.

```
// the same filter is applied to Customers, Sales, Inventories, StockMovements, ...
modelBuilder.Entity<Product>().HasQueryFilter(e => e.ShopId == CurrentShopId);
// No current shop (not signed in) -> the filter matches NOTHING: a forgotten shop fails closed.
```

Code that legitimately crosses shops (login looks an email up across all shops; token validation; startup tools) opts out explicitly with `IgnoreQueryFilters()`. That is the only place it appears. SKU, barcode and customer phone are unique *per shop*, so two shops can both stock COLA-500. Verified with two registered shops in the integration tests and on real PostgreSQL: one shop could not see, change, sell or delete another's data.

12. Idea 2: how loyalty points are calculated

A completed bill that has a **customer attached** earns:

```
points = floor( bill total / Loyalty:SpendPerPoint )      (default 100: 1 point per 100 spent)
```

- **Bill total** is what the customer actually pays: after the discount, with tax (the bill's TotalAmount).
- **Always rounded down**: a part-point is never awarded.
- **Walk-in bills** (no customer) earn nothing, and a bill that fails (say, out of stock) awards nothing.

Bill total	Points	Note
99.99	0	Just under one point's worth.
100.00	1	Exactly one.
456.75	4	3 x 145.00 + 5% tax; 4.5675 rounds down.
3,076.50	30	A real bill in the dev database.
250.00 at rate 50	5	If SpendPerPoint were set to 50.

Where the points live, and why it cannot drift

- Each bill stores `LoyaltyPointsEarned`; the customer's `LoyaltyPoints` is the running balance.
- Both are written **in the same database transaction as the sale**. The balance is updated with an atomic `balance = balance + n` (not read-modify-write), so two bills for one customer at the same moment cannot lose points. Eight concurrent bills were tested: the balance matched exactly.
- So the balance always equals the sum of its bills' points and can be re-derived from them.
- The POS shows the same formula as a preview ('earns 4 on this bill'), then the real result and new balance after the sale.
- Existing bills were given points by the migration's one-time backfill (at the default rate); demo history uses the same rule function.

Not built: spending (redeeming) points, expiry, tiers, and reversing points on a cancelled bill (there is no cancel feature yet).

13. Idea 3: email checks and verification

Proving someone owns an *email* can be done for free; proving they own a *phone* needs a paid SMS service, so only email is verified. At sign-up there are two stages.

Stage 1: before the account exists

- **Throwaway-mailbox domains** (mailinator, guerrillamail, 10minutemail and similar; a short built-in list) are refused. A DNS check alone cannot catch them, because those domains do have mail servers.
- **Can the domain receive mail?** The API asks Google Public DNS over HTTPS (free, no key) for MX records, falling back to an address record as email standards allow. A domain that does not exist is refused with 'check the address for typos'. **Only the domain is sent**, never the part before the @. Definitive answers are cached for an hour.
- **Fail open.** If the lookup fails (no internet, timeout), sign-up is allowed and a warning is logged. A DNS hiccup must never lock a real person out. The price: on a bad connection a made-up domain can slip through. This really happened once in testing.

Stage 2: proof of ownership

- After sign-up a **single-use link** is emailed: `/verify-email?token=<userId>.<secret>`, valid 24 hours.
- The secret is 32 random bytes. **Only its SHA-256 hash is stored**, compared in constant time; the token itself is never in the database.
- A new link replaces the old one (the old one stops working). Using a link clears the hash, so it works once. Forged, wrong-user or expired tokens all get the same 400.
- The app shows an amber banner with **Resend link** until the address is confirmed. **Verification is not enforced** yet: an unconfirmed account can use everything.

Sending the email

Set `Email:Smtp:*` to any SMTP server (keep the password in user-secrets). With no SMTP host, in Development the whole email is printed to the API log so you can copy the link; in any other environment nothing is sent and nothing sensitive is logged. Verified with a real SMTP connection to a local test server, with the message decoded as a mail client would.

14. Two complete flows

Register and verify

- **Register page** posts to /api/auth/register. The server normalises the email, checks it is unused, rejects throwaway or undeliverable domains, hashes the password, and saves the shop and user in one transaction.
- In Development it also seeds the sample catalogue (and demo history), generates a verification token, stores its hash, and emails the link. It returns a JWT, so the user is signed in immediately.
- The app shows the dashboard with the amber banner. Opening the emailed link calls /api/auth/verify-email; the server checks the hash and expiry, marks the address verified and clears the token. The banner disappears.

Checkout (with a customer)

- The cart sends only product ids, quantities, discount %, payment method, optional amount paid and optional customer. **No prices.**
- The server loads prices from the database, rejects unavailable products and too-large discounts, calculates totals and per-line shares, and works out change.
- In one transaction: for each line (in product-id order) the conditional stock update runs; if any returns nothing, everything rolls back with a 409. Otherwise it inserts the sale, items and ledger rows, then adds the loyalty points to the customer.
- The response is the full bill: totals, lines, change, points earned and the new balance. The POS shows them and offers Print.

15. Problems found and how they were fixed

Most of these were invisible to the in-memory tests and only appeared against the real database or in a browser, which is why verification used both.

Problem	Fix
LIKE ... ESCAPE '\' crashed on MySQL	Search used a backslash escape the MySQL provider mishandled. Changed to '!' (now also valid everywhere).
Dates unreadable on MySQL	The Oracle MySQL provider could not read date columns as DateOnly. Fixed with an explicit converter; moot after moving to PostgreSQL's native date.
CHECK constraints failed on PostgreSQL	Unquoted column names are folded to lower case. All CHECK expressions now quote their columns.
Search was case-sensitive on PostgreSQL	LIKE is case-sensitive there. Both sides are now lower-cased.
Bill numbers collided across shops	The demo generator was seeded identically per shop, and BillNumber is unique database-wide; the second shop's history failed and rolled back. Now seeded per shop, with a test.
A rerun guard that did not guard	The backfill command checked the earliest movement, but its first run moves that back, so a second run passed. Now compares with the shop's own creation time. Found by rehearsing on a copy of the real database.
Charts replayed their animation	A transient zero width destroyed and rebuilt the SVG. Zero widths are now ignored.
Donut centre text overflowed	Switched to a compact figure and a smaller font.
DNS lookup slow or timing out	Took 1-4 s on this network. Added a one-hour cache, a 6 s timeout, and kept it fail-open.
Stale PostgreSQL lock file	The Homebrew service kept crashing on a leftover postmaster.pid naming an unrelated process; verified it was stale before removing that one file.
My own test-harness mistakes	Several apparent failures were the test scripts (unparsed quoted-printable email, rate limit hit, timing waits). Each was traced to its cause rather than loosened.

16. What was verified, and what was not

Verified

Area	Result
Build	Release build: 0 warnings, 0 errors (warnings are treated as errors).
Backend tests	63 unit and 51 integration tests pass.
Frontend	22 Vitest tests pass; production build clean (initial bundle about 318 kB, 89 kB gzipped).
Migrations	Applied cleanly to an empty PostgreSQL database; the plain schema.sql also applies cleanly. The second migration was rehearsed on a copy of the real database, then applied to it.
API on PostgreSQL 16	A 70-check scripted run: registration, every endpoint, validation, arithmetic, per-line sums, cash change, customers, barcode search, rollback, case-insensitive search, and isolation between two shops. Twelve parallel sales for five units: exactly five succeeded, stock ended at zero.
Browser (headless Chrome)	Walkthroughs of registration, dashboard (desktop and phone widths, hover tooltips, period switching, no horizontal overflow, no console errors), products, customers, POS with customer, cash change and points, bill, inventory, reports, and the email banner and link.
Email, live	Real DNS lookups (a real domain accepted, a made-up one refused, a disposable one refused), delivery over a real SMTP connection, then opening the link.
Data integrity	After seeding history into the real dev database: stock equals the sum of movements, every movement starts where the previous ended, no negative stock, nothing dated in the future, loyalty balances equal the sum of bills' points; the user's real bills are unchanged.
Secrets	The API log was searched for the signing key, database password and a test password: none present.

Not verified

- Windows or Linux, and browsers other than Chrome.
- Sending to a real mailbox through a real provider (SMTP was tested against a local test server).
- A real printer. Dark mode (the app is light-theme only). Production deployment and a CI pipeline.
- Charts were checked by eye in screenshots; there are no automated visual tests.

17. What is left to do

Product features

- **Cancel and refund bills.** SaleStatus already has Cancelled; reports ignore it, but nothing sets it. A cancel should restock and reverse loyalty points.
- **Spend loyalty points** (redemption), expiry and tiers.
- **Staff accounts and permissions.** There is one login per shop owner; employees cannot have their own limited logins.
- **Password reset and change-password screens.** Only a development command exists today. Also email-based reset, and enforcing email verification for sensitive actions.
- **Phone verification** (needs an SMS provider).
- **Per-shop settings:** currency, time zone and tax rate are global; multi-currency and multiple tax rates per product.
- **Offline billing** (explicitly out of scope so far): an idempotency key on sales, a client queue and conflict handling when stock ran out meanwhile.
- Barcode-scanner hardware integration beyond typing into the search box; purchase orders and suppliers; per-line discounts and promotions; exports.

Engineering

- **CI** running both test suites plus a job against a real PostgreSQL service container, since SQLite cannot catch dialect problems.
- Move the JWT from localStorage to an HTTP-only cookie with refresh tokens (less exposed to script injection).
- Pre-aggregated daily tables if a shop's history grows beyond what in-memory bucketing handles comfortably (reports and insights bucket in memory for at most 366 and 180 days).
- A maintained disposable-domain list instead of the short built-in one; a second DNS provider as a fallback.
- Automated visual tests for the charts; a dark theme.
- Production deployment: HTTPS and HSTS, secrets in a vault, a separate database role for migrations, backups, structured logging and monitoring.

18. Running it

```
# one-time: database (PostgreSQL 14+)
psql postgres -c "CREATE ROLE smartpos LOGIN PASSWORD '<choose>'"
psql postgres -c "CREATE DATABASE smartpos_dev OWNER smartpos"

# one-time: configuration (secrets stay out of the repo)
cd smartpos/backend/src/SmartPOS.API
CS="Host=127.0.0.1;Port=5432;Database=smartpos_dev;Username=smartpos;Password=<pw>"
dotnet user-secrets set "ConnectionStrings:SmartPos" "$CS"
dotnet user-secrets set "Jwt:SigningKey" "$(openssl rand -base64 48)"
cd ../.. && dotnet tool restore
dotnet ef database update --project src/SmartPOS.Infrastructure --startup-project src/SmartPOS.API

# every day
cd smartpos/backend  && dotnet run --project src/SmartPOS.API --launch-profile http   # :5080
cd smartpos/frontend && npm install && npm start      # http://localhost:4200
# then open the app and choose "Create an account"; there is no seeded login

# development helpers (refuse to run outside Development)
dotnet run --project src/SmartPOS.API -- seed-demo-history --email you@example.com --days 90
dotnet run --project src/SmartPOS.API -- reset-password   --email you@example.com      # hidden prompt

# tests
cd smartpos/backend  && dotnet test SmartPOS.slnx
cd smartpos/frontend && npm test
```

Key settings

Setting	Meaning
Pos:TaxRatePercent / Pos:MaxDiscountPercent	Tax after discount; highest discount a bill may carry.
Loyalty:SpendPerPoint	Amount spent per loyalty point (default 100).
Store:CurrencyCode / Store:TimeZoneId	Money format; what 'today' and an hour mean.
App:WebBaseUrl, Email:Smtp:*, Email:LogMessages	Verification links and how they are sent.
Database:SeedSampleData / SeedDemoHistory	Sample catalogue and fabricated history for new shops (Development only).
RateLimiting:LoginPermitsPerMinute	Attempts per IP per minute on the public auth endpoints.